SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**
SARANYA.R.A(192572052)

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 40%

Duration :1 Hour
QUESTIONS: 4 **Total Marks:40**

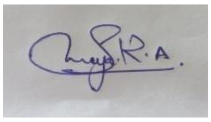
Annexure A

CASE STUDY

Code of Conduct:

I Saranya.R.A *(192572052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. Ax: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. Ax: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. Ax Ay: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO3 – ASSESSMENT TOOL 1
Case Study

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17
QUESTIONS: 4 **Duration:** 1 Hour **Total Marks:** 40

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Total Marks (40)
Understanding of Case	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/10
Application of Concepts	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/10
Analysis	20% (2)	/2	/2	/2	/2	/8
Solution Design	20% (2)	/2	/2	/2	/2	/8
Clarity	10% (1)	/1	/1	/1	/1	/4
Total per Question	**10 Marks**	**/10**	**/10**	**/10**	**/10**	**/40**

100

Case Study 1 — Smart Medical Diagnosis System

(a) Representation using Propositional Logic

Let the following propositional symbols represent the knowledge:

- **F** = Patient A has Fever
- **C** = Patient A has Cough
- **B** = Patient A has Breathlessness
- **Flu** = Patient A has Possible Flu
- **M** = Patient A has Possible Measles
- **P** = Patient A has Possible Pneumonia

Rules

Rule I:

Fever AND Cough → Possible Flu

[
F \land C \rightarrow Flu
]

Rule II:
Fever AND Rash → Possible Measles
Let **R** = Patient A has Rash.
[
F \land R \rightarrow M
]
Rule III:
Cough AND Breathlessness → Possible Pneumonia
[
C \land B \rightarrow P
]
Facts
For Patient A:
[
F = True
]
[
C = True
]
[
B = True
]
Therefore, the complete propositional knowledge base is:
[
KB = {F\land C\rightarrow Flu,;F\land R\rightarrow M,;C\land B\rightarrow P,;F,C,B}
]

(b) Forward Chaining
Forward chaining starts with known facts and repeatedly applies rules whose conditions are satisfied.
Initial Facts
[
F=True,\quad C=True,\quad B=True
]
Step 1 — Apply Rule I
Rule:
[
F\land C\rightarrow Flu
]
We know:
[

F=True
]
and
[
C=True
]
Therefore:
[
Flu=True
]
Derived fact: Patient A has Possible Flu.

Step 2 — Apply Rule II
Rule:
[
F\land R\rightarrow M
]
We know Fever is true, but there is **no fact that Rash is true**.
Therefore, Rule II cannot fire.
[
M \text{ cannot be derived}
]

Step 3 — Apply Rule III
Rule:
[
C\land B\rightarrow P
]
We know:
[
C=True
]
and
[
B=True
]
Therefore:
[
P=True
]
Derived fact: Patient A has Possible Pneumonia.

Forward Chaining Result

Step	Rule Fired	Facts Used	New Fact
1	Rule I	Fever, Cough	Possible Flu
2	Rule II	Fever, no Rash	No conclusion
3	Rule III	Cough, Breathlessness	Possible Pneumonia

Therefore, the possible diagnoses are:
[
\boxed{Possible\ Flu}
]
[
\boxed{Possible\ Pneumonia}
]
Possible Measles cannot be derived because Rash is not known to be true.

(c) Backward Chaining for Pneumonia

Goal
[
Goal = Pneumonia
]
Search the knowledge base for a rule that can conclude Pneumonia.
Rule III is:
[
C\land B\rightarrow P
]
Therefore, to prove Pneumonia, we must prove:
[
C
]
and
[
B
]

Goal Reduction Tree

```
          Pneumonia (P)
              |
        Rule III: C ∧ B → P
            /     \
           /       \
       Cough (C)   Breathlessness (B)
          |          |
```

FACT FACT
True True
Both subgoals are available as facts.
Therefore:
[
C=True \land B=True
]
implies:
[
P=True
]
Conclusion
[
\boxed{\text{Patient A has Possible Pneumonia}}
]
The backward-chaining proof succeeds because both required premises, Cough and Breathlessness, are directly available in the knowledge base.

Case Study 2 — Autonomous Traffic Management Agent
(a) Unification of Rule 1
Rule 1
[
\forall x;(Vehicle(x)\land EmergencyType(x)\rightarrow ClearPath(x))
]
The corresponding facts are:
[
Vehicle(Ambulance)
]
[
EmergencyType(Ambulance)
]
The variable in Rule 1 is:
[

x
]
We match the rule with the facts.
Unification Step 1
Match:
[
Vehicle(x)
]
with:
[
Vehicle(Ambulance)
]
Therefore:
[
\theta_1={x/Ambulance}
]
Unification Step 2
Match:
[
EmergencyType(x)
]
with:
[
EmergencyType(Ambulance)
]
This gives:
[
\theta_2={x/Ambulance}
]
Both substitutions are consistent.
Most General Unifier
[
\boxed{\theta={x/Ambulance}}
]
Applying this substitution to Rule 1:
[
Vehicle(Ambulance)\land EmergencyType(Ambulance)
\rightarrow ClearPath(Ambulance)
]
Since both premises are true:
[

\boxed{ClearPath(Ambulance)}
]
is derived.

(b) Resolution Proof of Proceed(CarA)
Relevant Facts

1.

[
Vehicle(Ambulance)
]

2.

[
EmergencyType(Ambulance)
]

3.

[
Vehicle(CarA)
]

4.

[
Behind(CarA,Ambulance)
]

Convert Rules to CNF
Rule 1
[
Vehicle(x)\land EmergencyType(x)\rightarrow ClearPath(x)
]
Eliminate implication:
[
\neg(Vehicle(x)\land EmergencyType(x))\lor ClearPath(x)
]
Using De Morgan's law:
[
\boxed{\neg Vehicle(x)\lor\neg EmergencyType(x)\lor ClearPath(x)}
]
Rule 2
[
ClearPath(x)\rightarrow GreenSignal(x)
]
One relevant CNF clause is:
[

\boxed{\neg ClearPath(x)\lor GreenSignal(x)}
]
Rule 2 also gives:
[
\neg ClearPath(x)\lor\neg RedSignal(x)
]
The second clause is not required to prove Proceed(CarA).
Rule 3
[
Vehicle(x)\land Behind(x,y)\land GreenSignal(y)\rightarrow Proceed(x)
]
CNF:
[
\boxed{\neg Vehicle(x)\lor\neg Behind(x,y)\lor\neg GreenSignal(y)\lor Proceed(x)}
]

Negate the Goal
Goal:
[
Proceed(CarA)
]
For resolution refutation, add:
[
\boxed{\neg Proceed(CarA)}
]

Resolution Step 1
From Rule 1:
[
\neg Vehicle(x)\lor\neg EmergencyType(x)\lor ClearPath(x)
]
Using:
[
Vehicle(Ambulance)
]
and
[
EmergencyType(Ambulance)
]
we derive:
[

\boxed{ClearPath(Ambulance)}
]

Resolution Step 2
Use Rule 2:
[
\neg ClearPath(x)\lor GreenSignal(x)
]
with:
[
ClearPath(Ambulance)
]
Therefore:
[
\boxed{GreenSignal(Ambulance)}
]

Resolution Step 3
Rule 3:
[
\neg Vehicle(x)\lor\neg Behind(x,y)\lor\neg GreenSignal(y)\lor Proceed(x)
]
Substitute:
[
x=CarA,\quad y=Ambulance
]
giving:
[
\neg Vehicle(CarA)\lor
\neg Behind(CarA,Ambulance)\lor
\neg GreenSignal(Ambulance)\lor
Proceed(CarA)
]
Using the facts:
[
Vehicle(CarA)
]
[
Behind(CarA,Ambulance)
]
and:

[
GreenSignal(Ambulance)
]
we derive:
[
\boxed{Proceed(CarA)}
]

Final Resolution
We have:
[
Proceed(CarA)
]
and the negated goal:
[
\neg Proceed(CarA)
]
Resolving them gives:
[
\boxed{\square}
]
The empty clause means contradiction, so the original goal is proven.
[
\boxed{Proceed(CarA)\text{ is true}}
]

(c) Handling Two Simultaneous Emergency Vehicles
The current knowledge base can represent multiple emergency vehicles because the rules are universally quantified:
[
\forall x
]
However, the current facts only explicitly identify one emergency vehicle:
[
Ambulance
]
To handle two simultaneous emergency vehicles, additional facts are required.
For example, suppose:
[
Vehicle(Ambulance1)
]

[
EmergencyType(Ambulance1)
]
[
Vehicle(Ambulance2)
]
[
EmergencyType(Ambulance2)
]
Then Rule 1 can independently derive:
[
ClearPath(Ambulance1)
]
and:
[
ClearPath(Ambulance2)
]
Rule 2 then derives:
[
GreenSignal(Ambulance1)
]
and:
[
GreenSignal(Ambulance2)
]

Additional rules that would improve the system

A priority rule could be added, for example:
[
\forall x;(EmergencyVehicle(x)\land HighPriority(x)
\rightarrow PriorityRoute(x))
]
A conflict-management rule could also be introduced:
[
\forall x\forall y; (EmergencyVehicle(x)\land EmergencyVehicle(y)
\land Conflict(x,y)\rightarrow ManageConflict(x,y))
]

Conclusion

The existing FOL structure supports multiple emergency vehicles, but the KB needs:

1. Facts identifying each emergency vehicle.
2. Their emergency types or priorities.
3. Their positions and relationships.

4. Rules for route conflicts and signal prioritization.

Therefore, the current KB is **structurally extensible but not sufficient by itself for complete simultaneous-emergency management**.

Case Study 3 — Agricultural Expert Reasoning System

(a) Conversion into CNF

Let:

- (S) = SoilDry
- (I) = IrrigationNeeded
- (W) = CropWheat
- (D) = ApplyDripMethod
- (R) = CropAtRisk

P1

Original:

$$
S\rightarrow I
$$

Step 1: Eliminate implication

$$
\neg S\lor I
$$

This is already CNF.

$$
\boxed{\neg S\lor I}
$$

P2

Original:

$$
I\land W\rightarrow D
$$

Step 1: Eliminate implication

$$
\neg(I\land W)\lor D
$$

Step 2: Move negation inward

Using De Morgan's law:

$$
(\neg I\lor\neg W)\lor D
$$

]
Therefore:
[
\boxed{\neg I\lor\neg W\lor D}
]

P3
Original:
[
\neg D\land W\rightarrow R
]
Step 1: Eliminate implication
[
\neg(\neg D\land W)\lor R
]
Step 2: Move negation inward
[
D\lor\neg W\lor R
]
Therefore:
[
\boxed{D\lor\neg W\lor R}
]

Facts
SoilDry is true:
[
\boxed{S}
]
CropWheat is true:
[
\boxed{W}
]
Complete CNF Knowledge Base
[
\boxed{\neg S\lor I}
]
[
\boxed{\neg I\lor\neg W\lor D}
]
[

\boxed{D\lor\neg W\lor R}
]
[
\boxed{S}
]
[
\boxed{W}
]

(b) Resolution Refutation of ApplyDripMethod
Goal
[
D
]
Negate the goal:
[
\boxed{\neg D}
]
Add (\neg D) to the CNF knowledge base.

Step 1
Clause:
[
\neg S\lor I
]
Fact:
[
S
]
Resolve them:
[
\boxed{I}
]
Therefore:
[
IrrigationNeeded
]
is true.

Step 2
Use:

[
\neg I\lor\neg W\lor D
]
We have:
[
I
]
and:
[
W
]
Resolve with (I):
[
\neg W\lor D
]
Resolve with (W):
[
\boxed{D}
]
Therefore:
[
ApplyDripMethod
]
is derived.

Step 3 — Contradiction
We added the negated goal:
[
\neg D
]
We now have:
[
D
]
and:
[
\neg D
]
Resolving them gives:
[
\boxed{\square}

]
Therefore, the theorem is proven.
[
\boxed{ApplyDripMethod}
]

Resolution Proof Tree

```
                    □
                  /   \
            ApplyDrip    ¬ApplyDrip
                 |
        -----------
        |            |
     IrrigationNeeded    CropWheat
        |
    SoilDry
      |
    Fact: S
```

A more detailed clause flow is:

```
    ¬S ∨ I          S
       \           /
        \ Resolve /
             ↓
             I

    ¬I ∨ ¬W ∨ D
      I       W
       \     /
        \   /
          D

          D       ¬D
           \     /
            \   /
              □
```

Hence:
[
\boxed{KB\models ApplyDripMethod}
]

(c) Removing SoilDry

Suppose the fact:

[
S
]
is removed.
The remaining facts are:
[
W
]
The rules remain:
[
\neg S\lor I
]
[
\neg I\lor\neg W\lor D
]
[
D\lor\neg W\lor R
]
Step 1
Without (S), we cannot resolve:
[
\neg S\lor I
]
to obtain:
[
I
]
Therefore:
[
IrrigationNeeded
]
cannot be derived.
Step 2
P2 requires:
[
I\land W
]
We have (W), but we do not have (I).
Therefore:
[
ApplyDripMethod

cannot be derived.

Step 3 — Can CropAtRisk be derived?

P3 is:

$$\neg D\land W\rightarrow R$$

To derive (R), we need:

$$\neg D$$

and:

$$W$$

We know:

$$W=True$$

But the absence of (D) does **not** logically mean:

$$\neg D$$

Therefore, (CropAtRisk) also cannot be derived.

Important Logical Point

Not proving (ApplyDripMethod) is different from proving:

$$\neg ApplyDripMethod$$

Therefore:

$$\boxed{CropAtRisk\text{ does not follow}}$$

Final Result

Knowledge	**With SoilDry**	**Without SoilDry**
SoilDry	True	Unknown
IrrigationNeeded	Derived	Not derived
CropWheat	True	True
ApplyDripMethod	Derived	Not derived
¬ApplyDripMethod	Not derived	Not derived

Knowledge	**With SoilDry**	**Without SoilDry**
CropAtRisk	Not derived	Not derived

Thus, removing SoilDry breaks the reasoning chain leading to IrrigationNeeded and ApplyDripMethod, but it **does not automatically imply CropAtRisk**.

Case Study 4 — Wumpus World Knowledge Agent
(a) Belief State and Modus Ponens

The agent receives the following percepts:

1. Stench at ([1,2])
2. Breeze at ([1,1])
3. Glitter at ([2,2])

Let:

- (S_{1,2}) = Stench at [1,2]
- (B_{1,1}) = Breeze at [1,1]
- (G_{2,2}) = Glitter at [2,2]
- (W_{adj(1,2)}) = Wumpus is adjacent to [1,2]
- (Pit_{adj(1,1)}) = Pit is adjacent to [1,1]
- (Gold_{2,2}) = Gold is at [2,2]

Step 1 — Stench

Rule:
[
Stench[1,2]\rightarrow WumpusAdjacent[1,2]
]
Fact:
[
Stench[1,2]
]
By Modus Ponens:
[
\boxed{WumpusAdjacent[1,2]}
]
Therefore, the Wumpus is in one of the cells adjacent to [1,2].

Step 2 — Breeze

Rule:
[
Breeze[1,1]\rightarrow PitAdjacent[1,1]
]
Fact:

[
Breeze[1,1]
]
By Modus Ponens:
[
\boxed{PitAdjacent[1,1]}
]
Therefore, a pit is in one of the cells adjacent to [1,1].

Step 3 — Glitter
Rule:
[
Glitter[x,y]\rightarrow Gold[x,y]
]
Given:
[
Glitter[2,2]
]
Therefore:
[
\boxed{Gold[2,2]}
]
The agent knows that gold is at [2,2].

Step 4 — Safe Cell Rule
Rule:
[
\neg Wumpus[x,y]\land\neg Pit[x,y]\rightarrow Safe[x,y]
]
However, the agent does not have explicit negative facts such as:
[
\neg Wumpus[x,y]
]
and:
[
\neg Pit[x,y]
]
Therefore, this rule cannot currently fire.
Belief State
The agent can conclude:
[

\boxed{WumpusAdjacent[1,2]}
]
[
\boxed{PitAdjacent[1,1]}
]
[
\boxed{Gold[2,2]}
]
But it cannot conclude that any particular cell is safe without additional negative information.

(b) Forward Chaining for Possible Wumpus and Pit Locations
Assuming the standard four-directional adjacency used in Wumpus World:
Wumpus
The percept is:
[
Stench[1,2]
]
Rule:
[
Stench[1,2]\rightarrow WumpusAdjacent[1,2]
]
The adjacent cells to [1,2] are:
[
[1,1], [1,3], [2,2]
]
assuming these cells exist in the world.
Therefore, the possible Wumpus locations are:
[
\boxed{[1,1], [1,3], [2,2]}
]

Pit
The percept is:
[
Breeze[1,1]
]
Rule:
[
Breeze[1,1]\rightarrow PitAdjacent[1,1]
]

The adjacent cells are:
[
[1,2], [2,1]
]
Therefore, the possible pit locations are:
[
\boxed{[1,2], [2,1]}
]

Glitter
The percept:
[
Glitter[2,2]
]
directly gives:
[
\boxed{Gold[2,2]}
]
Forward-Chaining Summary
Stench[1,2]
    |
    ↓
Wumpus adjacent to [1,2]
    |
    ↓
Possible Wumpus:
[1,1], [1,3], [2,2]

Breeze[1,1]
    |
    ↓
Pit adjacent to [1,1]
    |
    ↓
Possible Pits:
[1,2], [2,1]

Glitter[2,2]
    |

↓
Gold[2,2]

(c) Safety of Path [1,1] → [1,2] → [2,2]
The proposed path is:
[
[1,1]\rightarrow[1,2]\rightarrow[2,2]
]
Cell [1,1]
There is a Breeze at [1,1].
This means a pit may be adjacent to [1,1].
The adjacent cells include:
[
[1,2], [2,1]
]
Therefore, [1,1] itself is not proven to contain a pit by the given rule.
However, the knowledge base does not explicitly prove:
[
\neg Wumpus[1,1]
]
or:
[
\neg Pit[1,1]
]
So [1,1] is not formally established as safe by Rule 4.

Cell [1,2]
There is a Stench at [1,2].
Therefore, a Wumpus is somewhere adjacent to [1,2].
More importantly, the Breeze at [1,1] means a pit may be at [1,2].
Thus:
[
Pit[1,2]
]
is a possible location.
Therefore, [1,2] cannot be guaranteed safe.

Cell [2,2]
The Glitter tells us:
[
Gold[2,2]

]
However, the Stench at [1,2] makes [2,2] a possible Wumpus location.
Therefore:
[
Wumpus[2,2]
]
is possible according to the given knowledge.
So [2,2] also cannot be guaranteed safe.

Final Evaluation
The path:
[
\boxed{[1,1]\rightarrow[1,2]\rightarrow[2,2]}
]
is **not proven safe** by the current knowledge base.
The main reasons are:

- [1,2] is a possible pit location because of the Breeze at [1,1].
- [2,2] is a possible Wumpus location because of the Stench at [1,2].
- The KB lacks explicit negative facts needed to trigger the Safe rule.

Therefore:
[
\boxed{\text{The agent should not logically assume that the complete path is safe.}}
]
The agent can identify the gold at [2,2], but **knowing the gold's location does not imply that the route to it is safe**.